

Práctica Guiada: Construcción y Experimentación con MLPs

Competencia por Equipos: ¿Quién logra las mejores métricas?

Aprendizaje Profundo

Objetivos de la práctica

- Construir redes multicapa (MLPs) desde cero utilizando PyTorch.
- Experimentar con diferentes arquitecturas y analizar overfitting y underfitting.
- Usar TensorBoard para visualizar curvas de pérdida y accuracy.
- Trabajar en equipos para competir por obtener el mejor rendimiento en validación.

Herramientas a utilizar

- PyTorch para la construcción de redes.
- TensorBoard con SummaryWriter para registrar métricas.
- Dataset: `fifa2021_training.csv`
- Funciones auxiliares provistas: `load_array`, `evaluate_loss`, `accuracy`, `train_epoch_ch3`, `Accumulator`.

Configuración inicial (todos los equipos)

Paso 1: Preparación del entorno

1. Instalar las librerías necesarias: `torch`, `torchvision`, `tensorboard`, `pandas`, `scikit-learn`.
2. Si trabajan en Google Colab, instalar y configurar `ngrok` para acceder a TensorBoard.
3. Crear un directorio `runs/` donde se guardarán los registros de TensorBoard.

Paso 2: Carga y preprocesamiento del dataset

1. Cargar el archivo `fifa2021_training.csv` usando `pandas`.
2. Quedarse solo con las columnas: `Height`, `Weight`, `Age`, `Sex`, `Position` y todas las columnas de habilidades (por ejemplo, `Acceleration`, `Agility`, etc.).
3. Aplicar codificación `one-hot` a la variable `Sex`.
4. Separar características (`X`) y etiqueta `Position`.
5. Dividir en entrenamiento (70 %) y prueba (30 %) usando `train_test_split`.
6. Convertir los cuatro dataframes resultantes en tensores PyTorch de tipo `torch.float32` (etiquetas a `long`).

Paso 3: Creación de DataLoaders

Usar la función `load_array` para crear iteradores con `batch_size = 32` tanto para entrenamiento como para prueba.

Construcción de modelos base (fase individual por equipo)

Cada equipo debe implementar tres MLPs usando `nn.Sequential`:

- **modelo_chico**: dos capas ocultas con 4 neuronas cada una.
- **modelo_medio**: dos capas ocultas con 16 neuronas cada una.
- **modelo_grande**: dos capas ocultas con 256 neuronas cada una.

Todas las redes deben tener una capa de salida con 4 neuronas (una por cada posición). Usar funciones de activación `ReLU` entre capas.

Entrenamiento y registro de métricas

Implementación de la función `train`

Cada equipo debe escribir una función `train` que reciba:

- `modelo (net)`
- iterador de entrenamiento y prueba
- `SummaryWriter` y un nombre para el modelo
- número de épocas (fijar en 500)

Dentro de la función:

- Usar `CrossEntropyLoss` como función de pérdida.
- Usar `Adam` como optimizador (parámetros por defecto).

- Por cada época, llamar a `train_epoch_ch3` para obtener pérdida y accuracy en entrenamiento.
- Evaluar pérdida en prueba con `evaluate_loss`.
- Calcular accuracy en prueba usando la función `accuracy` sobre el iterador completo.
- Registrar con `writer.add_scalars`:
 - Pérdida de entrenamiento vs pérdida de prueba
 - Accuracy de entrenamiento vs accuracy de prueba
- Imprimir cada 10 épocas el progreso (época, pérdida train, pérdida test).

Entrenamiento de los tres modelos

Ejecutar `train` para cada modelo. Asegurarse de usar diferentes subdirectorios en `runs/` (por ejemplo, `runs/chico`, `runs/medio`, `runs/grande`) para que TensorBoard los muestre por separado.

Visualización con TensorBoard

Iniciar TensorBoard apuntando al directorio `runs` y analizar las gráficas generadas. Identificar curvas que evidencien underfitting o overfitting.

Competencia por equipos: búsqueda de la mejor arquitectura

Reglas de la competencia

- Cada equipo puede proponer hasta 5 arquitecturas diferentes de MLP.
- Se permite variar:
 - Número de capas ocultas (entre 1 y 5).
 - Cantidad de neuronas por capa.
 - Funciones de activación (ReLU, Tanh, LeakyReLU, ELU).
 - Regularización: Dropout (tasa entre 0.0 y 0.5).
 - Learning rate (probar valores como 0.01, 0.001, 0.0001).
 - Normalización por lotes (BatchNorm).
- Se debe entrenar cada modelo durante 300 épocas (para acelerar la competencia).
- La métrica principal para la competencia es el **accuracy en el conjunto de prueba**.
- En caso de empate, se desempata con la pérdida en prueba (menor es mejor).

Procedimiento para cada equipo

1. Definir una estrategia: ¿buscan alta capacidad (riesgo de overfitting) o modelos simples (riesgo de underfitting)?
2. Implementar una función que permita crear modelos con parámetros configurables.
3. Para cada arquitectura:
 - Crear el modelo.
 - Entrenarlo usando la función `train` (adaptada para aceptar learning rate y dropout).
 - Registrar en TensorBoard con un nombre único.
 - Guardar el accuracy final de prueba.
4. Documentar en una tabla todas las arquitecturas probadas y sus métricas.

Entrega de resultados

Cada equipo debe presentar:

1. Tabla comparativa de arquitecturas con:
 - Número de capas y neuronas.
 - Funciones de activación.
 - Dropout usado.
 - Learning rate.
 - Accuracy en prueba.
 - Pérdida en prueba.
 - Observaciones sobre overfitting/underfitting.
2. Capturas de TensorBoard mostrando las curvas de pérdida y accuracy.
3. Conclusión sobre cuál fue la mejor arquitectura encontrada y por qué.
4. Breve análisis de la relación entre complejidad del modelo y sobreajuste.

Actividad opcional: barrido sistemático

Para equipos que terminen antes o quieran profundizar, realizar un barrido de complejidad:

- Entrenar 10 modelos con dos capas ocultas, donde el número de neuronas por capa sea 2^k con $k = 0, 1, 2, \dots, 10$.
- Almacenar la pérdida final de entrenamiento y prueba para cada uno.
- Graficar en escala logarítmica la pérdida vs número de neuronas.
- Identificar visualmente el punto donde comienza el overfitting (la pérdida de prueba empeora mientras que la de entrenamiento sigue mejorando).

Rúbrica de evaluación (por equipo)

- **Correcta implementación (30 %)**: carga de datos, modelos base y función `train`.
- **Uso de TensorBoard (15 %)**: registros correctos y análisis de curvas.
- **Exploración de arquitecturas (30 %)**: cantidad y variedad de modelos probados.
- **Rendimiento en competencia (15 %)**: accuracy en prueba alcanzado.
- **Análisis y conclusiones (10 %)**: interpretación de overfitting/underfitting.

Preguntas guía para la discusión final

1. ¿Por qué un modelo demasiado pequeño tiene alta pérdida tanto en entrenamiento como en prueba?
2. ¿Qué forma tienen las curvas de pérdida cuando hay overfitting?
3. ¿El dropout siempre mejora el rendimiento de prueba? ¿Cuándo empeora?
4. ¿Qué relación observaron entre la cantidad de neuronas y la diferencia entre pérdida de entrenamiento y prueba?
5. ¿Cómo podrían saber si necesitan más datos o un modelo más complejo?

Recomendaciones finales

- Revisar siempre que los tensores tengan los tipos correctos (float para X, long para Y).
- No olvidar usar `net.train()` y `net.eval()` si implementan Dropout o BatchNorm.
- Explorar gradualmente: comenzar con arquitecturas simples antes de probar redes muy grandes.
- Competir sanamente: el objetivo es aprender, no solo ganar.

¡Que comience la competencia y el mejor modelo gane!